

On Demand Webinar: Unlocking documentation ROI with Google Cloud's DORA Lead → [Watch now](#)

Product ▾

How it
works
Swimm
platform
overview

Documents
Developer 1st
documentation

Integrations
Connect
your
existing
stack

Use
Cases ▾

Legacy
code
Modernize
&
maintain

Documenta
platform
Documenta
for enterpris

All posts

AI TOOLS FOR DEVELOPERS

AI Code Review: How It Works and 5 Tools You Should Know

Swimm Team 10 min read



On this page:

Related articles:

On this page:

Key Components of AI Code
Review
Advantages of AI for Code Review
Limitations and Concerns with AI-
Based Code Review
Popular AI Code Review Tools



What Is AI Code Review?

Artificial Intelligence (AI) code review is an automated process that examines the code of a software application for potential problems and inefficiencies. It involves the use of machine learning models to identify and fix coding errors, optimize code performance, and make recommendations for improvements.

The power of AI code review lies in its speed and scalability. It can analyze vast amounts of code in seconds, identifying patterns and anomalies that would be difficult, if not impossible, for a human to spot. Moreover, it can do this consistently and accurately, without the fatigue or bias that can affect human reviewers.

This innovative approach to code review is transforming the way developers write and refine code, making it possible to produce higher quality software, faster than ever before.

This is part of a series of articles about [AI tools for developers](#).

Related articles:

[Replit Ghostwriter vs. Copilot: 5 Key Differences and How to Choose](#)

[AI Coding Tools: Key Capabilities and 10 Tools You Should Know](#)

[7 Sourcegraph Cody Alternatives You Should Know](#)

[Codeium vs. GitHub Copilot: 4 Key Differences and How to Choose](#)

Key Components of AI Code Review

Static Code Analysis

Static code analysis involves examining the code without executing it, in order to identify potential issues such as syntax errors, coding standards violations, and security vulnerabilities.

Static code analysis is particularly useful for large and complex codebases. It can quickly scan through thousands of lines of code, pinpointing potential issues and providing detailed reports. AI algorithms can then use this information to make recommendations for improvements.

Dynamic Code Analysis

Unlike static code analysis, dynamic code analysis involves executing the code and observing its behavior. This allows the AI to identify runtime errors, performance issues, and other problems that may not be apparent from the code itself.

Dynamic code analysis is crucial for understanding how the code interacts with external systems and resources. It provides a more complete picture of the code's behavior, enabling the AI to make more informed and accurate recommendations.

Rule-Based Systems

Rule-based systems use a set of predefined rules to analyze the code and identify potential issues. They provide a consistent and reliable baseline for code analysis. They ensure that the code adheres to established coding standards and best practices. A common example of rule-based systems is a linter, which examines code for syntax errors or deviations from coding style guidelines.

Natural Language Processing (NLP) Models

NLP models are at the heart of AI code review. These models are trained on large datasets of code, learning to recognize patterns and anomalies that indicate potential problems or inefficiencies.

Over time, these models become increasingly adept at identifying and fixing issues in the code. They can

even learn from the feedback and corrections of human reviewers, continuously improving their performance.

Large Language Models (LLMs)

Code review tools are starting to incorporate Large language models (LLMs), like GPT-4. These models can understand the structure and logic of the code more deeply than traditional machine learning techniques, which helps in identifying more nuanced anomalies and errors.

Moreover, LLMs can also generate human-like comments and explanations for the code, making the review process more understandable and accessible to developers. This can be especially useful for novice developers who are still learning and can benefit from the detailed feedback provided by these models.

Furthermore, LLMs are language agnostic, meaning they can work with virtually any programming language. This makes them a versatile tool in code review processes, capable of handling diverse codebases and contributing to more efficient and thorough reviews.

Advantages of AI for Code Review

Efficiency and Speed

Traditional code review processes can be time-consuming and labor-intensive, often requiring multiple reviewers to examine the same code. With AI code review, the process can be completed in a fraction of the time. AI can quickly scan through the code, identify potential issues, and make recommendations for improvements.

Consistency and Accuracy

Human reviewers can be influenced by fatigue, bias, or other factors, leading to inconsistent or inaccurate reviews. AI, on the other hand, can analyze the code consistently and accurately, regardless of the size or complexity of the codebase. This ensures that more issues are identified and addressed, improving the overall quality of the software.

Detection of Hard-to-Find Errors

AI code review is also highly effective at detecting errors that are difficult to spot through manual review, either because they are subtle or because they only occur under certain conditions.

By analyzing the code in depth and considering a wide range of scenarios, AI can identify these elusive errors and suggest fixes. This reduces the risk of bugs slipping through the cracks and causing problems down the line.

Enhanced Learning and Skill Development

AI code review can serve as a valuable learning tool for developers. By providing detailed feedback and recommendations on a wider variety of coding issues and errors, and providing this feedback instantly as developers are coding, AI can help developers improve their skills and learn new techniques.

Related content: Read our guide to AI code generation (coming soon)

Limitations and Concerns with AI-Based Code Review

Over-Reliance on AI Tools

One of the most significant concerns with AI code review is the potential for over-reliance on these tools. As AI becomes more sophisticated, there is a danger that developers will start to depend on these tools too much, neglecting their own judgment and expertise. This can lead to a lack of understanding of the underlying code, making it more difficult to debug and maintain.

Moreover, while AI code review tools can help identify potential issues, they cannot replace a developer's understanding and intuition. AI can't fully understand the business logic behind the code or the specific requirements of a project. Therefore, while these tools can be a valuable aid, they should not be seen as a replacement for human developers.

Limitations in Understanding Context and Intent

Another limitation of AI code review is its inability to understand context and intent. When a developer

writes code, they do it with a specific intention in mind. They are trying to solve a particular problem or implement a specific feature. However, an AI code review tool can only analyze the code in a mechanical way, without understanding the broader context.

This lack of context can lead to false positives, where the tool flags code as problematic when it isn't. It can also miss issues that a human reviewer would spot because they understand the intent behind the code. Therefore, while AI code review can be a helpful tool, it still has a long way to go before it can fully replace human code review.

Handling of False Positives and False Negatives

False positives occur when a code review tool flags code as problematic when it's not, while false negatives occur when the tool misses actual issues.

False positives can be particularly frustrating, as they can lead to unnecessary work and can undermine confidence in the tool. On the other hand, false negatives can be even more problematic, as they can lead to bugs and security vulnerabilities being missed.

To overcome these challenges, AI code review tools need to continuously learn and improve. This can be achieved through machine learning algorithms that learn from past mistakes, as well as through feedback from developers.

Tips from the expert



Omer Rosenbaum

CTO & Co-founder at Swimm

In my experience, here are tips that can help you better leverage AI code reviews:

- 1. Pair AI reviews with human insights:** Use AI code reviews alongside human reviews. While AI can catch many issues, it doesn't understand the intent behind the code like a person does. Combining both helps ensure your code is both technically sound and aligns with your project goals.
- 2. Customize rules to fit your project** Don't just use the default settings. Adjust the AI tool's rules to

match your project's coding standards. This way, the tool gives you more useful feedback that's specific to your needs, reducing unnecessary alerts.

- 3. Use AI feedback as a learning tool:** Treat AI suggestions as a way to learn and improve. When AI flags something, discuss it with your team to understand why and how to avoid similar issues in the future. This turns code reviews into a great opportunity for growth and skill-building.

Popular AI Code Review Tools

Explore these popular tools for initiating AI code reviews or enhancing the review process.

Swimm



Swimm is a documentation and knowledge-sharing platform that complements the AI code review process. Swimm offers immediate access to detailed, context-rich documentation and codebase-specific knowledge directly within the IDE, helping developers understand enterprise codebases faster than ever before. Swimm enhances code reviews by providing improved context, auto-generated documentation, and consistency checks, ensuring alignment with best practices and facilitating knowledge transfer. Its AI assistant, /ask, delivers targeted answers to code-specific questions, streamlining problem-solving and reducing the time spent searching for information.

Swimm stands out with its focus on knowledge sharing and collaboration, enabling teams to set standards and share best practices. Its user-friendly interface and interactive tutorials make onboarding new members efficient and align the team with project goals. Overall, Swimm enhances code quality and team productivity by providing the necessary context for informed decisions.

In essence, Swimm complements AI code review tools by providing the contextual understanding necessary to make informed decisions, enhancing both code quality and team productivity.

Codacy



Codacy is an AI-powered code review tool that provides automatic code review for more than 30 languages. The platform integrates with GitHub, Bitbucket, and GitLab, enabling developers to find and fix issues directly from their repositories.



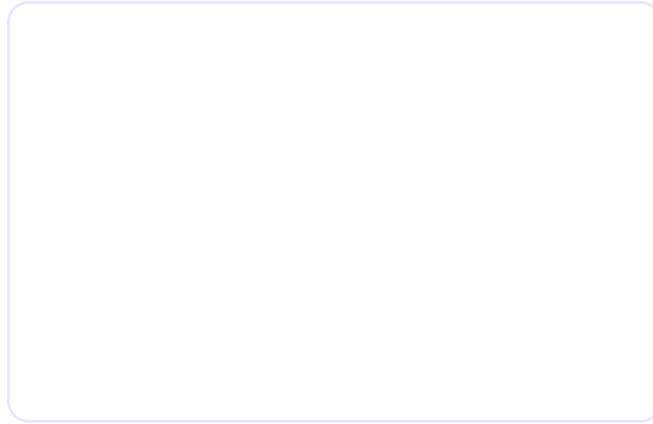
Codacy's AI engine can identify code patterns, detect bugs, identify security vulnerabilities and code duplication. It also offers a feature that allows developers to define their quality standards and enforce them across the team. This means that Codacy not only helps in maintaining code quality but also ensures consistency across the team's coding style and practices.

What sets Codacy apart is its user-friendly interface and dashboards. They provide visual insights into your codebase, highlighting areas of improvement and tracking your progress over time.

DeepCode



DeepCode is an AI code review tool that uses machine learning algorithms to learn from millions of software development repositories. This large dataset allows DeepCode to provide highly accurate suggestions and find potential issues that human reviewers might overlook.



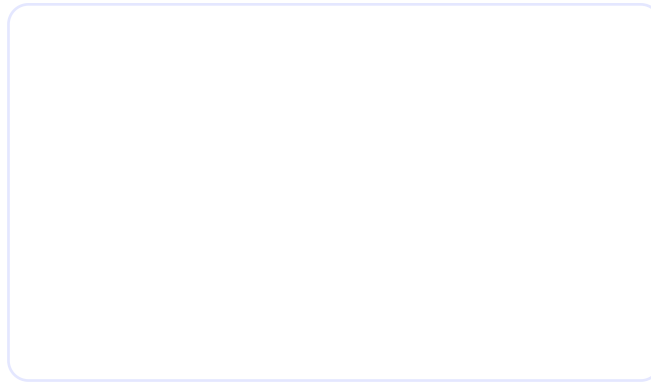
DeepCode supports multiple programming languages, including Java, JavaScript, TypeScript, Python, and C++. It can analyze your code in real-time, providing instant feedback and highlighting potential problems. In addition to spotting errors, DeepCode suggests solutions, making it useful for developer education.

One of the key features of DeepCode is its ability to detect subtle, hard-to-find bugs, security vulnerabilities, and performance issues.

Code Climate



Code Climate is another AI code review tool that can help you evaluate the maintainability of your code. Code Climate analyzes your codebase and assigns a maintainability score, giving you a quick overview of your code's health.



Code Climate supports a wide range of programming languages and integrates with GitHub, Bitbucket, and GitLab. Its AI engine identifies complex code, duplication, and potential bugs in real-time. It also offers automated code review comments, directly highlighting areas of concern on your pull requests.

Code Climate focuses on long-term code health. It provides insights into technical debt, helping teams understand the potential future impact of their coding decisions and improve the maintainability of their code.

Bito AI



Bito is an AI code assistant tool that provides developers with relevant feedback. Implement Bito in your IDE or CLI. Employing multiple prompts, Bito mirrors the role of specialized engineers, reviewing programs for security, performance, and structural integrity.

Using natural language prompts, Bito generates code in any programming language. The AI engine offers resources for command syntaxes, code explanations, or technical concepts in real-time. Bito integrates with GitHub and GitLab, with plans to expand to other hosting solutions.

Supporting various aspects of the development process, Bito AI can improve developer knowledge and strengthen codebases, giving valuable time back to engineers.

AI Code Reviews with Swimm

In a world where the pace of coding is accelerating, traditional code review methods are becoming less feasible. AI code review tools, as explored above, can offer remarkable benefits like rapid analysis, consistent accuracy, and the ability to pinpoint elusive errors. However, they are not without their limitations.

This is where Swimm comes into the picture. Swimm complements AI code review tools by addressing the need for human touch and context. Its code-coupled documentation ensures that your code is not just syntactically correct, but also understood. The benefit? A smoother onboarding process for new developers, quicker knowledge transfer among team members, and ultimately, a more efficient and better-aligned team.

Ultimately, AI code review tools have their role, but they can't be your whole strategy. Add Swimm to your toolchain to ensure that the machine's speed and scalability are complemented by the depth of human understanding, for a codebase that's not just error-free but also built for the future.

[Learn more about Swimm](#)

Compare plans

Explore our plans and pricing to see which is right for you.

See plans

Contact sales

Get in touch with a Swimm expert to discuss your needs.

Pick a time

On-demand demo

See an in-depth overview of Swimm.

Watch now

Get started

- Book a demo
- Our GitHub app
- Our VS Code plugin
- Our JetBrains plugin

Product

- How it works
- Document
- Integrations

Resources

- Blog
- Learning center
- Doc site
- Customer stories

Use cases

- Legacy code
- Documentation platform
- AI-ready code
- Technology service providers

Company

- Partner Po
- Pricing
- About us
- Careers
- Community